TD6: Programmation dynamique et algorithmes gloutons – Correction

CPES « Sciences des données, arts et cultures »

Antoine GAUQUIER
antoine.gauquier@ens.fr

Pierre ABOULKER
pierreaboulker@gmail.com

14 janvier 2025

Dans ce TD, il vous est proposé de résoudre deux exercices types TD afin de vous préparer à l'examen : un sur la programmation dynamique et un sur les algorithmes gloutons.

Empilement de tuiles

Question 1

Deux critères majeurs justifient l'utilisation de la programmation dynamique :

— Sous-structures optimales : il est possible de déterminer la solution du problème à partir de solutions optimales du sous-problème. On comprend bien ici qu'on peut déterminer le nombre de tours différentes avec (n, m, k) en comptant le nombre de tours obtenues dans des « sous-configurations » : $(n, m-1, k)$, $(n-1, m-1, k)$, etc.

— Sous-problèmes répétés : certains sous-problèmes vont être rencontrés de multiples fois. En effet, pour déterminer le nombre de tours avec (n, m, k), il va falloir déterminer le nombre de tours que l'on peut réaliser avec des largeurs de tuiles et des hauteurs de tours inférieures de façon récursive, qui mènera à la rencontre de sous-problèmes communs.

Question 2

La stratégie repose sur les caractéristiques du problème : il n'est possible de poser une nouvelle tuile que sur une tuile autant ou plus large. Ainsi, pour s'assurer de ne pas rater de combinaisons, il faut partir de la tuile la plus large, et progressivement diminuer la largeur des tuiles.

Attention toutefois : il est possible d'utiliser k fois chaque largeur de tuile. Or, comme ces tuiles doivent être posées par ordre décroissant de largeur, alors des tuiles de même largeur sont nécessairement empilées les unes sur les autres.

Ainsi, pour déterminer le nombre de tuiles avec une hauteur de n, une largeur maximale de h et un nombre maximal de tuiles de même largeur de k, il suffit de compter le nombre de tours qui peuvent être construites :

— En ne choisissant aucune tuile de largeur m. Cela consiste à résoudre le même problème avec les paramètres $(n, m-1, k)$.

— En choisissant une seule tuile de largeur m. Cela consiste à résoudre le même problème avec les paramètres $(n-1, m-1, k)$.

— En choisissant plusieurs tuiles de largeur m, bien entendu sans jamais dépasser k tuiles de la même largeur. En notant l ce nombre de tuiles de largeur m choisies, cela revient à résoudre le même problème avec les paramètres $(n-l, m-1, k)$

La formule de récursivité est alors :

$$N(n, m, k) = \sum_{l=0}^{\min(k,n)} N(n-l, m-1, k)$$

, $\min(n, k)$ permettant de traiter le cas éventuel où $k > n$.

Deux cas d'arrêt assurent la bonne terminaison, et surtout l'assurance d'obtenir le nombre de tours attendu (correction). Il faut d'abord identifier la situation dans laquelle on a obtenu une tour complète. Cette situation a lieu lorsque l'on a déjà ajouté n tuiles à la tour, et donc au moment de l'appel $N(0, m, k)$ pour n'importe quel m et k. Ainsi, $N(0, m, k) = 1$ (on peut aussi l'interpréter comme le fait qu'il n'y a qu'une façon de construire une tour de hauteur 0 : c'est la tour vide). Il faut enfin arrêter d'essayer de construire une tour non-vide lorsque plus aucune tuile n'est disponible. Cela se traduit concrètement pas $N(n, 0, k) = 0$ pour tout n et k.

Ceci nous donne finalement :

$$N(n, m, k) = \begin{cases} 1 & \text{si } n = 0 \\ 0 & \text{si } m = 0 \\ \sum_{l=0}^{\min(k,n)} N(n-l, m-1, k) & \text{sinon} \end{cases}$$

Question 3

Les algorithmes 1 et 2 présentent le pseudo-code d'une solution faisant appel à la mémoïsation. L'algorithme 3 présente le pseudo-code d'une solution sans mémoïsation.

Pour rappel : lorsque l'on utilise la mémoïsation, on part du problème à résoudre et l'on fait appel aux sous-problèmes de façon récursive (on parle aussi de résolution *top-down*), alors que sans, on part des cas de base et on remplit une structure de données avec les solutions de sous-problèmes intermédiaires jusqu'à résoudre le problème souhaité (on parle dans ce cas de résolution *bottom-up*).

Algorithme 1 Algorithme pour le problème de l'empilement de tuiles avec mémoïsation (tours_memoisation)

Entrée: n la hauteur des tours, m la largeur maximale des tuiles et k le nombre maximal de tuiles par valeur de largeur.
Sortie: $N(n, m, k)$ le nombre de tours différentes pour les paramètres passés en entrée.

```
memo ← Tableau bidimensionnel de taille (n + 1) × (m + 1)

pour i ∈ {0, ..., n} faire
    pour j ∈ {0, ..., m} faire
        memo[i, j] ← −1
    fin pour
fin pour

Retourner tours_memoisation_recursif(n, m, k, memo)
```

Algorithme 2 Algorithme récursif pour le problème de l'empilement de tuiles avec mémoïsation (tours_memoisation_recursif)

Entrée: n la hauteur des tours, m la largeur maximale des tuiles et k le nombre maximal de tuiles par valeur de largeur. `memo`, la structure de mémoïsation.
Sortie: $N(n, m, k)$ le nombre de tours différentes pour les paramètres passés en entrée.

```
si memo[i, j] ≠ −1 alors
    Retourner memo[i, j]
fin si

si n = 0 alors
    Retourner 1
sinon si m = 0 alors
    Retourner 0
sinon
    nb_tours ← 0
    pour l ∈ {0, ..., min(n, k)} faire
        nb_tours ← nb_tours + tours_memoisation_recursif(n − l, m − 1, k, memo)
    fin pour
    memo[n, m] ← nb_tours
    Retourner nb_tours
fin si
```

Algorithme 3 Algorithme récursif pour le problème de l'empilement de tuiles sans mémoïsation (tours_memoisation_iteratif)

Entrée: n la hauteur des tours, m la largeur maximale des tuiles et k le nombre maximal de tuiles par valeur de largeur.

Sortie: $N(n, m, k)$ le nombre de tours différentes pour les paramètres passés en entrée.

```
memo ← Tableau bidimensionnel de taille (n + 1) × (m + 1)

pour i ∈ {0, ..., n} faire
    pour j ∈ {0, ..., m} faire
        memo[i, j] ← 0
    fin pour
fin pour

pour i ∈ {0, ..., m} faire
    memo[0, i] ← 1
fin pour

pour i ∈ {0, ..., n} faire
    pour j ∈ {0, ..., m} faire
        nb_tours ← 0
        pour l ∈ {0, ..., min(n, k)} faire
            nb_tours ← nb_tours + memo[n − l, m − 1]
        fin pour
        memo[i, j] ← nb_tours
    fin pour
fin pour

Retourner memo[n, m]
```

Question 4

L'analyse de l'algorithme 3 se fait rapidement :

— La ligne 1 se fait dans le pire des cas en $O(n \times m)$.

— Les lignes 2 à 6 présentent deux boucles pour imbriquées, la première itérant n fois et la seconde m fois. Ce qui est exécuté pour chacune des $n \times m$ itérations est en $O(1)$: le tout est en $O(n \times m)$.

— Les lignes 7 à 9 présentent une unique boucle pour à m itérations, ici aussi avec des opérations en $O(1)$: le tout est en $O(m)$.

— Les lignes 10 à 18 présentent trois boucles pour imbriquées. La première fait n itérations, la deuxième m et la troisième $\min(n, k)$. Les opérations au sein des boucles sont toutes en $O(1)$: le tout est en $O(\min(n, k) \times n \times m)$.

Le total est donc en $O(n \times m) + O(n \times m) + O(m) + O(\min(n, k) \times n \times m) = O(\min(n, k) \times n \times m)$. Ainsi on obtient $O(k \times n \times m)$ si $k < n$ (la situation la plus probable), et $O(n^2 \times m)$ sinon.

La complexité temporelle est en $O(n \times m)$, à savoir la taille du tableau créé pour enregistrer les valeurs qui ont déjà été calculées.

La complexité temporelle de l'algorithme récursif avec mémoïsation sera la même : il s'agit dans les deux cas de la même approche avec la même formule de récurrence.

Arrêts de bus

Question 5

Le nombre d'arrêts minimal dans ce cas de figure est 2, et une solution optimale est $A^* = (a_1, a_2) = (280, 490)$.

On peut vérifier que la solution est valide :

— $d(x_1, a_1) = 100 \leqslant 100$

— $d(x_2, a_1) = 80 \leqslant 100$

— $d(x_3, a_1) = 90 \leqslant 100$

— $d(x_4, a_2) = 100 \leqslant 100$

— $d(x_5, a_2) = 100 \leqslant 100$

Voir correction en classe pour le dessin, mais il s'agit de tracer un segment allant de 0 à 590 où l'on place les maisons et arrêts selon leur positions x_i ou a_i, respectivement.

Question 6

Une bonne stratégie consiste à placer le premier arrêt le plus loin possible de la position de la première maison, tout en s'assurant qu'elle reste désservie par ledit arrêt. En procédant ainsi, on maximise la « rentabilité » de l'arrêt, car il couvrira au moins autant mais éventuellement plus de maisons que s'il était placé plus proche de la première maison. Concrètement, il s'agit de choisir $a_1 = x_1 + \ell$. On continue ainsi pour toutes les maisons de la rue qui ne seraient pas déjà desservies par un arrêt précédent.

Question 7

Algorithme 4 Algorithme glouton pour le choix des arrêts de bus

Entrée: ℓ la distance maximale séparant une maison d'un arrêt et $X = (x_1, \ldots, x_n)$ les distances des n maisons au début de la rue.

Sortie: m le nombre d'arrêts de bus créés et A^* la liste de leurs positions dans la rue.

1: $Z \leftarrow -\infty$ (La zone couverte par les arrêts)
2: $m \leftarrow 0$ (Le nombre d'arrêts créés)
3: $A^* \leftarrow$ Liste vide qui contiendra la position des arrêts

4: **pour** $i \in \{1, \ldots, n\}$ **faire**
5: **si** $x_i > Z$ **alors**
6: $m \leftarrow m + 1$
7: $a_m \leftarrow x_i + \ell$
8: Ajouter a_m dans A^*
9: $Z \leftarrow a_m + \ell$
10: **fin si**
11: **fin pour**

12: **Retourner** (m, A^*)

Question 8

Les lignes 1, 2, 3 et 12 se font en temps constant $O(1)$. La ligne 4 indique une boucle pour où l'on itère n fois. Dans le pire des cas, on rentre dans le si de la ligne 5 à chaque itération, et les lignes 6 à 9 se font toutes en temps constant. La complexité temporelle de l'algorithme est donc en $O(n)$.

L'algorithme présenté ne fonctionne que si les distances X des maisons au début de la rue sont triées par ordre croissant, sans quoi nous n'avons pas la garantie que l'algorithme retourne une solution correcte. Ainsi si la liste n'était pas triée, il faudrait le faire en amont, et la complexité passerait en $O(n \log n)$ (la complexité temporelle standard des algorithmes de tri efficaces).

Question 9

On considère la solution $\texttt{ALG} = (b_1, \ldots, b_k)$, la solution retournée par l'algorithme associé au critère de gloutonnerie décrit (qui correspond à l'algorithme 4). D'autre part, on écrit $\texttt{OPT} = (c_1, \ldots, c_m)$ une solution optimale, i.e., une solution qui minimise m et dessert les n maisons tel que décrit dans l'énoncé.

La solution OPT est choisie comme suit : parmi toutes les solutions optimales (partant du principe qu'il en existe plusieurs ; le cas échéant, il n'y a pas de choix à faire), on choisit celle

qui maximise $i \in [\![1, n]\!]$ tel que les i premiers arrêts de ALG sont à la même position que les i premiers de OPT. En d'autres termes, on aura $(b_1, \ldots, b_i) = (c_1, \ldots, c_i)$, mais $b_{i+1} \neq c_{i+1}$.

Il s'agit alors maintenant de déterminer les formes possibles de OPT en fonction des valeurs de i. On divise le problème en trois cas de figures :

1. $\underline{i > k}$

Ce cas de figure est impossible. ALG et OPT ne peuvent pas partager davantage d'éléments que ALG n'en contient ($|\texttt{ALG}| = k$).

2. $\underline{i = k}$

Dans un tel cas, OPT contient tous les éléments de ALG. Or ALG retourne une solution correcte au problème, et l'objectif est de minimiser le nombre d'arrêts à créer. De ce fait, on a forcément $m = k$, sans quoi OPT n'est pas optimal (puisqu'il existera une solution correcte ALG au problème qui contiendra moins d'arrêts que OPT). Cela implique donc que dans ce cas de figure, la solution ALG est elle aussi optimale.

3. $\underline{i < k}$

Comme les positions des i premiers arrêts sont les mêmes pour ALG et pour OPT, alors ils desservent exactement le même nombre de maisons. Notons ce nombre p. Ainsi, la $(p+1)$–ème maison n'est pas desservie par les i premiers arrêts. Il faudra alors créer un nouvel arrêt pour la desservir. ALG le fera en choisissant $b_{i+1} = x_{p+1} + \ell$, d'après notre critère de gloutonnerie.

On a alors l'assurance que $c_{i+1} \leqslant b_{i+1}$. En effet, b_{i+1} est la distance maximale à laquelle on peut placer le $(i+1)$–ème arrêt sans que la $(p+1)$–ème maison ne soit plus desservie. En d'autres termes, choisir $c_{i+1} > b_{i+1}$ c'est placer le $(i+1)$–ème arrêt à plus de ℓ mètres de la $(p+1)$–ème maison qui ne serait alors pas desservie.

Ainsi, étant donné qu'on a forcément $c_{i+1} \leqslant b_{i+1}$, ALG avec ses $(i+1)$ premiers arrêts desservira au moins autant de maisons que OPT, si ce n'est plus. On peut donc construire une solution $\texttt{OPT}' = (\texttt{OPT} \setminus c_{i+1}) \cup b_{i+1} = (c_1, \ldots, c_i, b_{i+1}, c_{i+2}, \ldots, c_m)$, qui est non seulement correcte mais également optimale. En effet, les $(c_{i+2}, \ldots, c_m)$ derniers arrêts desservent par définition toutes les maisons qui ne seraient éventuellement pas desservies par les $(c_1, \ldots, c_i, b_{i+1})$ premiers, et on se retrouve avec une solution de taille $|\texttt{OPT}'| = |\texttt{OPT}| = m$ qui est minimale.

Ceci entre en conflit avec notre hypothèse de départ affirmant que $b_{i+1} \neq c_{i+1}$ et contredit donc le choix de OPT, montrant ainsi que ALG est une solution optimale également dans ce cas de figure. □